SGA — Sistema de Gestão de Alunos Neurodivergentes

Plano de Produção

O que precisa ser implementado e modificado
para o sistema ficar pronto para produção

Documento de *gap analysis* baseado na leitura do código atual do repositório `sga-neurodivergente` (backend Laravel 11 / PHP 8.3 / PostgreSQL 16 + frontend estático). Cobre segurança, portabilidade do ambiente Docker, integração *front*↔API, conformidade LGPD e correção de defeitos.

1 Sumário executivo

A versão atual do sistema foi construída para **demonstração** (defesa do Projeto Integrador), não para operação real. O backend em Laravel é funcional e bem estruturado como MVP acadêmico — tem criptografia de dados sensíveis via `APP_KEY`, trilha de auditoria, *soft delete*, consentimento LGPD e controle de vínculo monitora↔aluno. Porém, três lacunas o impedem de ir para produção:

As três lacunas estruturais

1. O frontend não conversa com o backend. O `frontend/js/script.js` é 100% estático: login conferido no próprio JavaScript, dados *mock* em `localStorage` e "criptografia AES-256" apenas simulada com `btoa()`. Nenhuma chamada HTTP é feita à API. Hoje são dois sistemas que não se falam.

2. Segurança de demonstração. `APP_DEBUG=true`, senhas de *seeder* fracas e versionadas (`admin123`), credenciais de banco no `docker-compose.yml`, ausência de *rate limiting*, de CORS e de HTTPS.

3. Portabilidade frágil do Docker. O ambiente só sobe de forma confiável na máquina onde foi montado, por causa de `composer update` em tempo de execução, portas fixas, imagens sem *pin* e, principalmente, a `APP_KEY` ser regenerada a cada ambiente — o que torna qualquer dado já cifrado ilegível em outra máquina.

Este documento detalha cada item, com prioridade, local no código e ação recomendada. A seção Seção 2 trata especificamente dos problemas de "rodar em outra máquina" que você observou.

Área	Situação atual (resumo)	Prioridade
Portabilidade Docker	`composer update` em runtime, portas fixas, imagens sem pin, APP_KEY volátil	CRÍTICA
Integração front↔API	Frontend estático; login e criptografia falsos; zero chamadas HTTP	CRÍTICA
Segurança / hardening	APP_DEBUG on, sem rate limit, sem CORS, sem HTTPS, segredos versionados	CRÍTICA
Gestão de chave (APP_KEY)	Chave no `.env`, regenerada por ambiente, sem rotação	CRÍTICA
Tratamento de erros	`withExceptions` vazio; stack trace exposto com debug on	MÉDIA
Defeitos de código	FK `monitora_id` NULL quando admin registra ocorrência; teste desalinhado	MÉDIA
LGPD — retenção/expurgo	PDFs de laudo não cifrados e não removidos no *soft delete*	MÉDIA
Paginação / busca	`GET /alunos` sem paginação; busca inviável em campos cifrados	MÉDIA
Observabilidade / backup	Sem monitoramento, sem política de *backup* do banco e dos arquivos	MÉDIA
CI/CD e qualidade	Sem pipeline; `factories/` vazio; cobertura parcial	BAIXA

Tabela 1: Visão geral das frentes de trabalho e prioridades.

2 Portabilidade do ambiente (rodar em outra máquina)

Esta é a causa direta dos problemas que você teve ao rodar fora da sua máquina, mesmo com Docker + Compose. Os pontos abaixo foram confirmados em `backend/docker-compose.yml`, `backend/docker/Dockerfile`, `backend/docker/entrypoint.sh` e `backend/.env.example`.

2.1 `composer update` em vez de `composer install`

Tanto o `Dockerfile` quanto o `entrypoint.sh` executam:

```
composer update --no-interaction --prefer-dist --no-audit 2>/dev/null || true
```

Três problemas de uma vez:

- **`update` ignora o `composer.lock`** → cada máquina (ou cada data) pode resolver versões diferentes das dependências. O build deixa de ser reproduzível.
- **`2>/dev/null || true`** → se a instalação falhar, o erro é engolido e o container "sobe" mesmo assim, sem as dependências. É por isso que em outra máquina pode subir mas não funcionar.
- **O volume `.:/var/www`** (no compose) monta o projeto do host por cima do container, sobrescrevendo o `vendor/` instalado no build. Se o host não tiver `vendor/`, tudo passa a depender do `composer update` do *entrypoint*.

> **Ação**
> Trocar por `composer install` (respeita o `.lock`), versionar o `composer.lock`, remover `2>/dev/null || true` para que falhas apareçam, e usar um *multi-stage build* que copie o `vendor/` para dentro da imagem — sem depender do volume do host para dependências. Em produção, montar o código via imagem (não via *bind mount*).

2.2 APP_KEY regenerada a cada ambiente

O `.env.example` tem `APP_KEY=` vazio e o `entrypoint.sh` roda `php artisan key:generate --force` em todo *boot* sem chave. Como os dados sensíveis (nome do aluno, ocorrências, laudos, planos de manejo) são cifrados com a `APP_KEY` (via `CriptografiaService` → `Crypt`), **trocar de máquina gera uma chave nova e torna qualquer dado cifrado existente ilegível**. A função `decifrarDados` ainda agrava: ao falhar, ela retorna o próprio texto cifrado em vez de erro, mascarando o problema.

Ação

Definir a `APP_KEY` como **segredo gerenciado** (variável de ambiente injetada pelo orquestrador, *secret manager* ou cofre), igual em todos os ambientes que compartilham o mesmo banco. Nunca regenerar em *boot*. Documentar que a chave é parte do *backup* (sem ela, não há como ler os dados). Planejar rotação de chave (re-cifrar em migração controlada). Fazer `decifrarDados` falhar de forma explícita em vez de devolver o *ciphertext*.

2.3 Portas fixas e colisão

O compose expõe portas fixas: `8000:80` (web), `5432:5432` (Postgres) e `8080:8080` (Adminer). Em outra máquina que já use o Postgres local na 5432 (muito comum) ou algo na 8000/8080, o `up` falha ou sobe parcialmente.

Ação

Parametrizar as portas com variáveis (`${APP_PORT:-8000}`, `${DB_PORT:-5432}`). Em produção, **não** publicar a porta do Postgres para o host (deixar só na rede interna do compose) e remover o Adminer do ambiente produtivo.

2.4 Imagens sem *pin* e sem plataforma definida

`php:8.3-fpm-alpine`, `postgres:16-alpine`, `nginx:alpine` e `adminer:latest` usam *tags* flutuantes, sem *digest* (`@sha256`). O compose também não fixa `platform:`. Em um Mac ARM (M1/M2/M3), as imagens x86 rodam sob emulação e as extensões PHP compiladas via `pecl`/`$PHPIZE_DEPS` podem divergir. `adminer:latest` pode mudar a qualquer momento.

Ação

Fixar *tags* específicas e, idealmente, *digests*. Definir `platform` ou publicar imagem *multi-arch*. Trocar `adminer:latest` por versão fixa (e removê-lo em produção).

2.5 Outros pontos de portabilidade

Ponto no código	Risco / ação
`entrypoint.sh`: `until nc -z db 5432` sem *timeout*	Loop infinito se o banco nunca subir; o container fica preso em "Aguardando Banco". Adicionar limite de tentativas e sair com erro claro.
`entrypoint.sh`: `chmod -R 777 storage bootstrap/cache`	Permissão insegura e, em volume do Windows, mapeamento de UID divergente. Ajustar *ownership* correto (usuário do php-fpm) em vez de 777.
`migrate --force` e `db:seed --force` em todo *boot*	Roda *seed* com usuários de demonstração automaticamente. Em produção, separar *migrate* (passo controlado) de *seed* (nunca em produção).
`#!/bin/bash` em imagem Alpine	Funciona porque o Dockerfile instala `bash`, mas é frágil. Padronizar para `sh` POSIX ou garantir `bash`.
`DB_HOST=db` fixo	Correto dentro do compose, mas quebra ao rodar `artisan` fora do container. Documentar / parametrizar.
`.env` não versionado, copiado de `.env.example`	Sobe com `APP_ENV=local`, `APP_DEBUG=true`, `LOG_LEVEL=debug`. Criar `.env.production` de referência com valores seguros.

Tabela 2: Demais ajustes de portabilidade no ambiente Docker.

3 Segurança e *hardening*

3.1 Configuração de ambiente insegura (padrão)

`APP_ENV=local`, `APP_DEBUG=true` e `LOG_LEVEL=debug` no `.env.example`. Com *debug* ligado e `withExceptions` vazio em `bootstrap/app.php`, qualquer exceção não tratada retorna *stack trace* completo ao cliente.

Ação
Em produção: `APP_ENV=production`, `APP_DEBUG=false`, `LOG_LEVEL=warning`. Implementar *handler* de exceções que retorne JSON padronizado (sem vazar *trace*).

3.2 Segredos versionados

`docker-compose.yml` e `.env.example` trazem `POSTGRES_PASSWORD=sgan_secret` em texto. Senha fraca e no repositório.

Ação
Mover credenciais para variáveis de ambiente / *secret manager*. Gerar senha forte por ambiente. Garantir que `.env` real nunca seja versionado (já está no `.gitignore`).

3.3 *Seeders* com senhas fracas

`UserSeeder` cria `admin@escola.edu.br / admin123`, `monitora@escola.edu.br / monitora123` (e variações `@instituicao.edu.br`), executados automaticamente pelo *entrypoint*.

Ação
Não rodar *seed* de usuários em produção. Criar o primeiro administrador por comando interativo (`artisan`) com senha forte, ou por processo de *bootstrap* controlado.

3.4 Ausência de *rate limiting*

Nenhuma rota tem `throttle` — nem o `POST /api/login`. Isso deixa o login exposto a força bruta.

Ação

Aplicar `throttle` no login (ex.: por IP + login) e um limite geral nas rotas autenticadas. Considerar bloqueio progressivo após N tentativas.

3.5 CORS e HTTPS

Não há `config/cors.php` no projeto (o front atual nem chama a API, então nunca foi necessário). O nginx (`docker/nginx/default.conf`) serve HTTP puro, sem TLS nem cabeçalhos de segurança.

Ação

Ao integrar o front: configurar CORS restrito à origem do front. Em produção: TLS (certificado via proxy/*load balancer* ou Let's Encrypt) e cabeçalhos de segurança (HSTS, X-Content-Type-Options, etc.).

4 Integração Frontend ↔ API

Hoje o `frontend/` e o `backend/` são **dois sistemas independentes**. O `frontend/js/script.js` não faz nenhuma requisição HTTP: o login é comparado literalmente em JavaScript, os dados vêm de constantes *mock* (`SEED_ALUNOS`, `SEED_OCORRENCIAS`) persistidas em `localStorage`, e a "criptografia AES-256" é encenada — gera um texto com `btoa(...)` e guarda os fatos em claro dentro de `(Fatos: ...)`, que outra função extrai de volta.

Ação — reescrever a camada de dados do front

- Substituir o login *hardcoded* por `POST /api/login`, guardando o *token* Sanctum e enviando-o como `Authorization: Bearer` nas demais chamadas.
- Trocar todo o acesso a `localStorage`/*mock* por chamadas reais aos *endpoints* (`/api/alunos`, `/api/ocorrencias`, `/api/laudos/...`, etc.).
- Remover a criptografia simulada — a cifra real já acontece no servidor.
- Remover atalhos de demonstração (`preencherCredenciais`, `simularPerfil`) e mensagens de banca.
- Tratar *token* expirado/401 (redirecionar ao login) e erros de rede.

5 Conformidade LGPD — pendências

A base de LGPD está montada (cifra, auditoria, consentimento, *soft delete*, portabilidade e vínculo monitora↔aluno), mas restam lacunas:

Pendência	Ação	Prior.
PDFs de laudo não cifrados em disco	Cifrar o arquivo em repouso ou usar *storage* com criptografia; manter fora de disco público (já está em `laudos_privados`).	MÉDIA
Laudos órfãos após *soft delete*	Rotina de expurgo que remove arquivos quando o aluno é apagado/retido; política de retenção documentada.	MÉDIA
`exportar-lgpd` sem checagem de vínculo	`AlunoController::exportarLgpd` deve validar que a monitora tem vínculo com o aluno (hoje qualquer autenticado exporta qualquer aluno).	CRÍTICA
Exportação só em JSON	Oferecer também PDF, conforme previsto na documentação do projeto (direito à portabilidade).	BAIXA
Logs de auditoria incompletos	Registrar também login/logout e acesso à lista `GET /alunos`.	BAIXA
CPF/e-mail de responsável em claro	Avaliar cifrar `responsaveis.cpf/email` como os demais dados pessoais.	MÉDIA

Tabela 3: Pendências de conformidade e proteção de dados.

6 Defeitos de código a corrigir

6.1 FK `monitora_id` nula ao registrar ocorrência como admin

A migration de `ocorrencias` define `monitora_id` como **NOT NULL** (`constrained`), mas `OcorrenciaController::store` grava `monitora_id => $monitora ? $monitora->id : null` quando quem registra é um admin sem registro de `Monitora`. No PostgreSQL isso viola a restrição → erro 500 no fluxo "admin registra ocorrência".

> **Ação**
> Decidir a regra de negócio: ou tornar `monitora_id` *nullable* (ocorrência pode não ter monitora), ou impedir que admin registre ocorrência, ou associar a ocorrência a uma monitora válida. Ajustar *migration* + *controller* + teste de forma coerente.

6.2 Teste desalinhado do schema

`LgpdAdequacaoTest::setUp` cria `Administrador` com `cargo` e `cpf`, campos que não existem no model (`fillable: [user_id, setor, nivel_acesso]`) nem na tabela `administradores`.

> **Ação**
> Alinhar o teste ao schema real (ou adicionar as colunas, se forem desejadas). Criar *factories* (hoje `database/factories/` só tem `.gitkeep`) para padronizar a montagem de dados nos testes.

6.3 `nivel_acesso` definido mas nunca usado

`administradores.nivel_acesso` é editado em `PermissaoController` mas não é verificado em nenhuma rota — a autorização real é só por `perfil` (string). Ou se implementa a verificação, ou se remove o campo para não dar falsa sensação de controle.

7 Escalabilidade e operação

Tema	Ação
Paginação	`GET /alunos` retorna todos os registros e decifra em memória. Adicionar paginação em alunos, ocorrências e auditoria.
Busca em campos cifrados	Como a cifra usa IV aleatório, não há `WHERE LIKE` nos campos sensíveis. Para buscar por nome, avaliar *hash* determinístico (*blind index*) ou campo de busca normalizado, com os devidos cuidados de LGPD.
Tratamento de erros	Preencher `withExceptions` com *handler* que padroniza respostas de erro e esconde detalhes internos.
Observabilidade	Centralizar *logs*, adicionar métricas e *health check* real (hoje só há `/up`).
Backup e restauração	Rotina de *backup* do PostgreSQL **e** do diretório de laudos **e** da APP_KEY; testar restauração. Sem a chave, o *backup* do banco é inútil.
Fila/jobs	`QUEUE_CONNECTION=database` configurado, mas sem *worker*. Se usar filas (ex.: expurgo, e-mails), subir um processo de *worker*.

Tabela 4: Itens de escalabilidade e operação contínua.

8 Roteiro sugerido

Fase	Escopo	Prioridade
1	Tornar o ambiente reproduzível: `composer install` + `.lock`, *pin* de imagens, portas parametrizadas, APP_KEY como segredo fixo, *timeout* no *wait* do banco.	CRÍTICA
2	*Hardening*: `APP_DEBUG=false`, *rate limiting* no login, segredos fora do repositório, *handler* de exceções, primeiro admin via comando.	CRÍTICA
3	Integração real do frontend com a API (login Sanctum, chamadas HTTP, remoção de *mock* e cripto falsa), CORS + HTTPS.	CRÍTICA
4	Correção de defeitos (FK de ocorrência, teste/factories), pendências LGPD (vínculo em export, expurgo de laudos).	MÉDIA
5	Operação: paginação/busca, *backup*/restauração, observabilidade, CI/CD.	MÉDIA

Tabela 5: Ordem recomendada de execução, da base para o topo.

Observação final

O backend tem uma base sólida — o esforço maior não é reescrever, e sim (a) estabilizar o ambiente para rodar igual em qualquer máquina, (b) ligar o frontend à API de verdade e (c) desligar os atalhos de demonstração. Feitas as fases 1 a 3, o sistema já se torna um produto real; as fases 4 e 5 o deixam robusto para uso contínuo.